Dynamic Arrays

DSA27 — Lecture 04 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 4 · Fall 2026

Where We Are

Two weeks ago, one problem

The course `Array` gives $O(1)$ indexing — and a **fixed size**.

```
a = Array(4)
a[4] = "e"          # IndexError: there is no slot 4, and there never will be
```

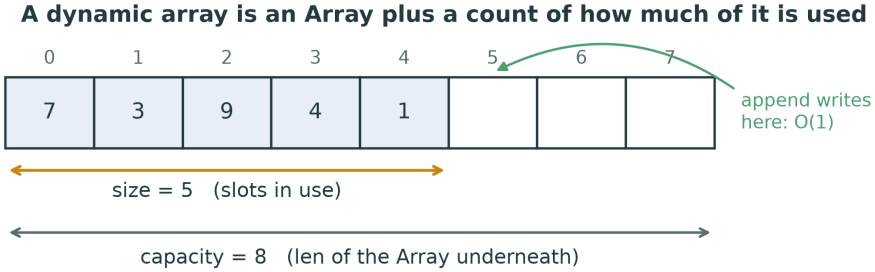
Python's `list` grows for ever, and append is fast. **How?**

Today

1. Capacity and size: an Array plus a count
2. Growing: allocate, copy, switch
3. **How much** to grow — by 1, by 100, or × 2?
4. **Amortised** analysis: why append is $O(1)$ on average
5. Shrinking, and how not to thrash
6. Python's own `list`, measured
7. The rest of the operations, and their costs

Capacity and Size

An Array plus a count



- **capacity** — how many slots the Array underneath has (`len(self._block)`)
- **size** — how many of them hold data (`self._size`, and what `len()` returns)
- Always $0 \leq \text{size} \leq \text{capacity}$

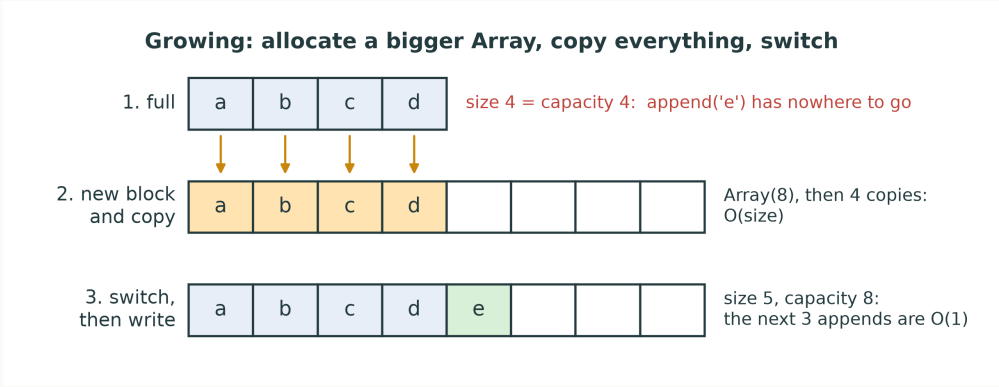
Reading and writing are still O(1)

```
def __getitem__(self, index):
    if index < 0:
        index += self._size                # -1 means the last USED slot
    if not 0 <= index < self._size:
        raise IndexError(index)
    return self._block[index]
```

Negative indices are **translated**: the course `Array` has none, so the dynamic array provides them — in $O(1)$.

Growing

Allocate, copy, switch



A full array cannot grow in place. So:

1. allocate a **new, bigger** `Array`
2. **copy** every element across — $O(\text{size})$
3. **switch** `self._block` to the new Array (the old one is garbage-collected)

How Much to Grow?

Option 1: one slot at a time

Capacity 1, 2, 3, 4, … — **every** append is a resize.

Copies for n appends:

$$0 + 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2} = \Theta(n^2)$$

Every append is $O(n)$. 100,000 appends ≈ 5 **billion** copies.

Option 2: a constant step — say, 100 at a time

Resize only every 100 appends — surely much better?

Copies for n appends: $100 + 200 + 300 + \cdots \approx \frac{n^2}{2 \times 100}$

Still $\Theta(n^2)$ — just 100 times smaller. 100,000 appends ≈ 50 million copies.

Option 3: double

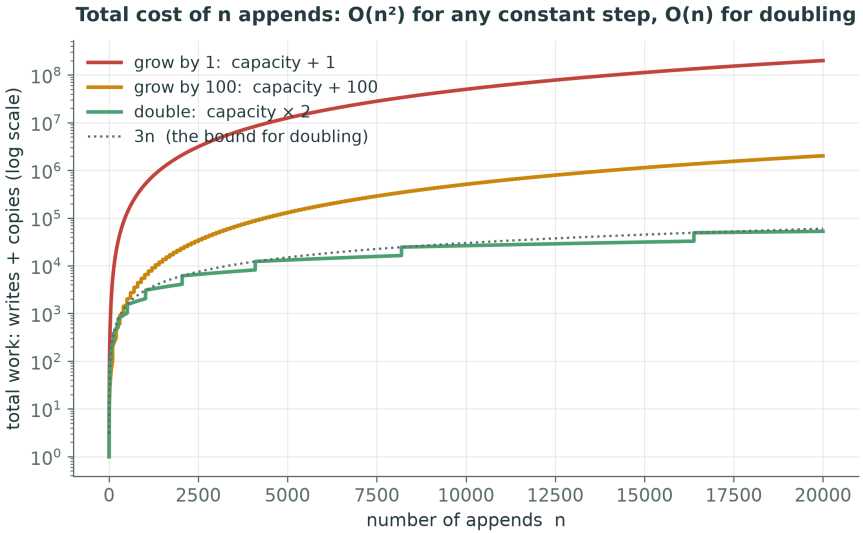
Capacity 1, 2, 4, 8, 16, … — resize only when size reaches a power of two.

Copies for n appends:

$$1 + 2 + 4 + \cdots + 2^k \;<\; 2n \qquad (2^k < n)$$

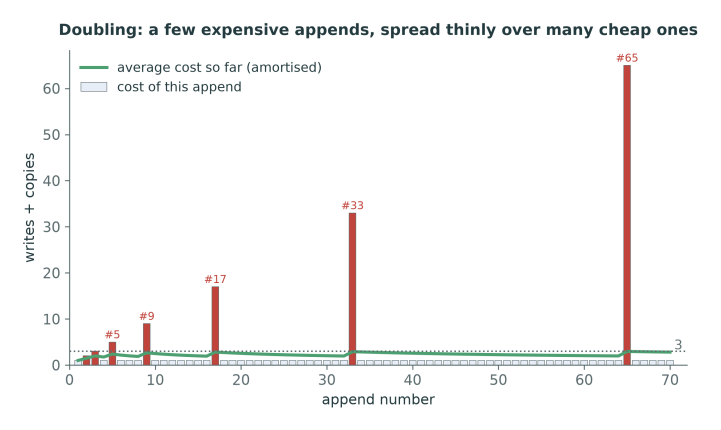
$\Theta(n)$ **in total.** 100,000 appends: 131,071 copies.

The three side by side



Amortised Analysis

One expensive append, many cheap ones



The red bars are resizes: expensive, but **exponentially rare**. The green line — the average so far — never rises above **3**.

Watch it run: the Array Lab, again

Array Lab — the *Dynamic array* scenarios.

- append from empty: watch the block **double** and every element copied
- A bar per append and the running average — the last slide's chart, built live
- `insert_at` on a full block, `pop()` against `pop(0)`, `da[-1]`
- **Goes wrong:** `_resize` that forgets to copy, or forgets to switch; growing by **one** slot (120 copies for 16 appends)

Amortised cost: the definition

The **amortised cost** of an operation is the **total cost of any sequence** of n operations, **divided by n**.

For append with doubling: total $< 3n$, so the amortised cost is $< 3 = O(1)$.

- One individual append can still cost $O(n)$ — the **worst case** is unchanged.
- But **no sequence** of n appends can cost more than $3n$ — a **guarantee**.

Amortised is not “average case”

	Average case (Lecture 02)	Amortised
Averages over	possible **inputs**	the operations in **one sequence**
Needs	an assumption about the inputs	nothing
Guarantee	“usually”	**always**, for every sequence

append is **worst-case** $O(n)$, **amortised** $O(1)$. Both statements are true and both are useful.

Any constant factor works

Growth rule	Total copies for n appends	Amortised append	Worst unused space
+ 1	$\approx n^2/2$	$O(n)$	none
+ c (constant)	$\approx n^2/2c$	$O(n)$	c slots
× 2	$< 2n$	$O(1)$	about half
× 1.5	$< 3n$ (roughly)	$O(1)$	about a third

What makes it $O(1)$ is growing by a **factor** — a percentage of the current size — not the particular factor.

Shrinking

Give memory back — carefully

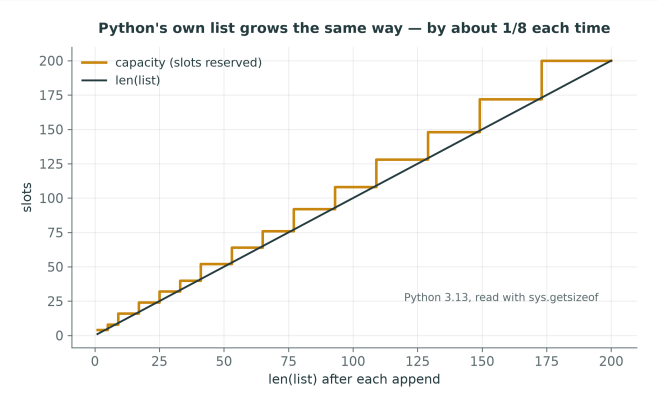
Pop until the array is nearly empty, and most of the Array is wasted. Shrink when size falls to **one quarter** of capacity — and then **halve**.

Why a quarter, not a half?

Start: capacity 8, size 8 (full)	halve at ½ full	halve at ¼ full
append → size 9	double to 16: copy 8	double to 16: copy 8
pop → size 8	**halve** to 8: copy 8	nothing (8 > 16/4)
append → size 9	**double** to 16: copy 8	nothing
pop → size 8	**halve** to 8: copy 8	nothing
repeat…	**every operation** $O(n)$	$O(1)$

Python's `list`

Measured: CPython does exactly this



Read straight from the interpreter with `sys.getsizeof`: capacity 4, 8, 16, 24, 32, 40, 52, 64, 76, 92, … — grow by about **one eighth**, plus a little.

All the Operations

`DynamicArray`: the contract with costs

Operation	Cost	Why
`a[i]`, `a[i] = x`, `len(a)`	$O(1)$	address arithmetic; a stored count
`append(x)`	**amortised** $O(1)$, worst $O(n)$	occasional resize, paid for by the rest
`pop()` from the end	$O(1)$	nothing moves
`insert_at(i, x)`	$O(n)$	shift `i .. size-1` right (and maybe resize)
`pop(i)` from the middle	$O(n)$	shift `i+1 .. size-1` left
`x in a`, search	$O(n)$	one by one
iterate	$O(n)$	

Exactly the costs of Python's `list` — because it **is** one.

What it is built for

From here on, the dynamic array is **storage** for other structures:

Week	Structure	Uses the `DynamicArray` as
6	`Stack`	the items — top at the **end**, so push and pop are $O(1)$
7	`SlowQueue`	the items — `pop(0)` from the front, $O(n)$: measure it
12	`MinHeap`	the complete binary tree, stored level by level
14	`Graph`	each node's list of neighbours

A bug in your `DynamicArray` becomes a bug in all four. **Test it well now.**

This Week

Exercises: `dsa/dynamic_array.py`

Method	Target	Watch out for
`append(value)`	amortised $O(1)$	resize **before** writing; grow by `self.growth`
`_resize(capacity)`	$O(n)$	copy only `size` elements; count `resize_count`
`__getitem__`, `__setitem__`	$O(1)$	bounds are `0 .. size-1`; translate negatives
`insert_at(index, value)`	$O(n)$	resize if full; shift from the **end**
`pop(index=-1)`	$O(1)$ at the end, else $O(n)$	`IndexError` when empty; clear the freed slot

```
pytest tests/test_dynamic_array.py -v
```

`test_doubling_keeps_resizes_logarithmic`: 1,000 appends must resize fewer than 20 times.

Homework 4 — before Lecture 05

1. **Implement** `dsa/dynamic_array.py` until `pytest tests/test_dynamic_array.py` passes.
2. **Measure.** Make a copy of your class that grows by `+ 10` instead of `× growth`. Time 1,000 … 32,000 appends for both with `viz.complexity.measure`, and plot with `reference=["n", "n^2"]`. Plot `resize_count` against n for both as well.
3. **Prove.** With growth factor 3 (capacity 1, 3, 9, 27, …), show that n appends cost $O(n)$ in total.
4. **Extend.** Add shrinking to your class: halve the capacity when pop leaves the array one quarter full. Write a test that alternates append and pop at the boundary, and check with `resize_count` that it does not thrash.

Practise online

- **Week 4 quiz**: 40 multiple-choice questions, every answer explained
- **Written practice** on the same page: 9 questions. Write, then mark yourself against the marking points
- **Lecture 4 test**: 20 questions, 25 minutes, timed
- **Midterm simulator**: Lectures 1–7, papers of up to 60 questions
- In Arabic: week 4 · midterm

Summary

Seven things to keep

1. A dynamic array is an **`Array` plus a size**; capacity ≥ size.
2. Full? **Allocate** a bigger Array, **copy** everything, **switch** — $O(n)$.
3. Grow by a **constant** → $\Theta(n^2)$ for n appends. Grow by a **factor** → $\Theta(n)$.
4. **Amortised** $O(1)$: any n appends cost < 3n. The worst single append is still $O(n)$.
5. Amortised is a **guarantee over a sequence**, not an average over inputs.
6. Shrink at **one quarter** full, not one half — or it **thrashes**.
7. Python's `list` is exactly this, growing by about **1/8**.

Next

Week 5 — Linked lists. The opposite trade: no copying, ever, and $O(1)$ insertion at the front — paid for with $O(n)$ indexing.